

Task-A Report

Team-Aurora

1 Problem Statement

The core objective of the *Deep Fake Comsys Hackathon 6 (Task A)* is to develop a robust audio forensic system capable of multi-class deepfake detection in challenging acoustic environments. Unlike traditional binary spoofing detection tasks, this challenge requires the model to disentangle intrinsic spoofing artifacts from extrinsic channel distortions.

Formally, given an input audio waveform $x \in \mathbb{R}^T$, the system must map x to one of four distinct categories $y \in \{0, 1, 2, 3\}$:

1. **Real (Class 0):** Clean, bona fide human speech.
2. **Real-Distorted (Class 1):** Bona fide speech degraded by channel noise, compression, or environmental artifacts.
3. **Fake (Class 2):** Synthetic speech generated via Text-to-Speech (TTS) or Voice Conversion (VC) algorithms.
4. **Fake-Distorted (Class 3):** Synthetic speech that has undergone further channel distortion.

2 System Architecture

We utilized the **AASIST** model, which is designed to model artifacts spanning heterogeneous spectral and temporal domains.

2.1 Backbone Components

- **RawNet2-based Encoder:** The model ingests raw waveforms directly. It utilizes a Sinc-convolution layer followed by six residual blocks to extract a high-dimensional feature map F [1].
- **Graph Module:** The feature map is split into two parallel graphs: a Spectral Graph ($\mathcal{G}_s$) and a Temporal Graph ($\mathcal{G}_t$).
- **Heterogeneous Stacking Graph Attention Layer (HS-GAL):** This novel component enables information flow between the spectral and temporal graphs using a stack node to accumulate heterogeneous information [1].
- **Readout:** The system employs max and average pooling followed by a linear output layer for 4-class classification.

2.2 Model Size

The model is lightweight, containing approximately **298,000 trainable parameters**.

3 Methodology

3.1 Preprocessing Pipeline

To standardize the variable input data, we applied the following preprocessing steps prior to training:

1. **Resampling:** All audio inputs were resampled to **16 kHz**.
2. **Mono Conversion:** Multi-channel audio was averaged to a single mono channel.
3. **Tensor Shaping:** We applied random padding or cropping to ensure a fixed input dimension of $1 \times 64,600$ samples (approx. 4 seconds).

3.2 Imbalance Handling

The dataset exhibited severe class imbalance. To mitigate this, we implemented **Weighted Cross-Entropy Loss**. We assigned class weights w_c inversely proportional to the class frequencies ($w_c \propto \frac{1}{N_c}$), which ensures that the model is penalized more heavily for misclassifying minority classes (e.g., Real samples). The weighted classification loss $\mathcal{L}$ for a given sample is formally defined as:

$$\mathcal{L}(x, y) = - \sum_{c=1}^C w_c \cdot \mathbb{I}(y = c) \cdot \log \left(\frac{\exp(x_c)}{\sum_{j=1}^C \exp(x_j)} \right) \quad (1)$$

where C is the total number of classes, x is the input vector of logits, y is the true class label, and $\mathbb{I}(\cdot)$ is the indicator function.

3.3 Sequential Training Strategy

We devised a 3-stage pipeline to progressively refine the model’s decision boundary.

3.3.1 Stage 1: Base Model 1

We trained the AASIST backbone from scratch on an **80% stratified partition** of the training data. This model serves as the stable baseline.

- **Duration:** 15 Epochs.
- **Objective:** Establish general feature extraction capabilities.

3.3.2 Stage 2: Base Model 2

We initialized a second model with the weights from Base Model 1 and continued training on the **full 100% dataset**. This creates a “Generalist” model that has seen all available data but retains the stability of the first stage.

- **Initialization:** Weights from Base Model 1.
- **Duration:** 15 Epochs on 100% data.

3.3.3 Stage 3: Hard Mined Model

We implemented a custom *Hard Negative Mining* loop .

1. **Inference:** We ran inference using Base Model 2 on the training set.
2. **Filtering:** We extracted a “Hard Subset” of samples based on two criteria:
 - **Misclassification:** The model predicted the wrong label.
 - **Low Confidence:** The maximum softmax probability fell within the ambiguity range of $[0.4, 0.7]$.
3. **Fine-Tuning:** A fresh model instance was fine-tuned specifically on this Hard Subset to resolve these ambiguities.

4 Experimental Setup

Table 1: Hyperparameter Configuration

Parameter	Value
Input Length	64,600 samples
Batch Size	32
Optimizer	Adam
Learning Rate	5×10^{-5}
Scheduler	Cosine Annealing
Loss Function	Weighted Cross-Entropy

5 Inference and Results

5.1 Ensemble Strategy

To maximize robustness and minimize variance, the final prediction is a weighted ensemble of the three models generated during the training pipeline. We applied weighted softmax averaging:

$$P_{final} = w_1 P_{M1} + w_2 P_{M2} + w_3 P_{M3} \quad (2)$$

Where:

- **M_1 (Base Model 1):** $w_1 = 0.35$
- **M_2 (Base Model 2):** $w_2 = 0.50$ (Highest weight due to exposure to 100% data).
- **M_3 (Hard Mined):** $w_3 = 0.15$ (Used as a corrective term for edge cases).

Table 2: Classification Performance Scores Across Classes

Class	Precision	Recall	F1-Score	Support
Class 0 (Real)	0.98	0.99	0.99	2512
Class 1 (RealDist)	0.99	0.99	0.99	10049
Class 2 (Fake)	1.00	1.00	1.00	6954
Class 3 (FakeDist)	1.00	1.00	1.00	27818
Accuracy			1.00	47333
Macro Avg	0.99	1.00	0.99	47333
Weighted Avg	1.00	1.00	1.00	47333

5.2 Key Observations

- **Weighted Loss Necessity:** Without inverse frequency weighting, recall for the “Real” class was significantly degraded. The weighted loss balanced the gradients effectively.
- **Turnout Effectiveness:** The hard mining stage successfully recovered samples that were consistently confusing for the base models, particularly in the Real-Distorted category.

6 Conclusion

We successfully adapted the AASIST architecture for deepfake detection on an imbalanced dataset. By moving from a standard training loop to a sequential transfer learning approach with targeted hard negative mining, we achieved a robust detection system capable of handling ambiguous audio artifacts.

References

- [1] J. Jung et al., “AASIST: Audio Anti-Spoofing Using Integrated Spectro-Temporal Graph Attention Networks,” in Proc. ICASSP, 2022.